

$A\alpha$	$B\delta$	$C\beta$	$D\varepsilon$	$E\gamma$
$B\beta$	$C\varepsilon$	$D\gamma$	$E\alpha$	$A\delta$
$C\gamma$	$D\alpha$	$E\delta$	$A\beta$	$B\varepsilon$
$D\delta$	$E\beta$	$A\varepsilon$	$B\gamma$	$C\alpha$
$E\varepsilon$	$A\gamma$	$B\alpha$	$C\delta$	$D\beta$

ТЕОРИЯ ГРАФОВ

Независимость и покрытия.

Ядро графа

Задача о наименьшем покрытии

Покрывающие множества вершин и рёбер

- Вершина покрывает инцидентные ей рёбра, а ребро *покрывает* инцидентные ему вершины.
- Множество вершин, которые в совокупности покрывают все рёбра, называется *вершинным покрытием*.
- Наименьшее число вершин во всех вершинных покрытиях называется *числом вершинного покрытия* и обозначается α_0 .

1. $\alpha_0(K_p) = p - 1$, если K_p — полный граф.
2. $\alpha_0(K_{m,n}) = \min(m, n)$, если $K_{m,n}$ — полный двудольный граф.
3. $\alpha_0(\overline{K}_p) = 0$, если $\overline{K}_p$ — вполне несвязный граф.
4. $\alpha_0(G_1 \cup G_2) = \alpha_0(G_1) + \alpha_0(G_2)$, если G_1 и G_2 — компоненты несвязного графа.

Покрывающие множества вершин и рёбер

- Множество рёбер, которые в совокупности *покрывают* все вершины, называется *рёберным покрытием*.
- Наименьшее число рёбер во всех рёберных покрытиях называется *числом рёберного покрытия* и обозначается α_1 .
- Для графа с изолированными вершинами α_1 не определено.

1. $\alpha_1(C_{2n}) = n$, если C_{2n} — чётный цикл.
2. $\alpha_1(C_{2n+1}) = n + 1$, если C_{2n+1} — нечётный цикл.
3. $\alpha_1(K_{2n}) = n$, если K_{2n} — полный граф с чётным числом вершин.
4. $\alpha_1(K_{2n+1}) = n + 1$, если K_{2n+1} — полный граф с нечётным числом вершин.
5. $\alpha_1(K_{m,n}) = \max(m, n)$, если $K_{m,n}$ — полный двудольный граф.

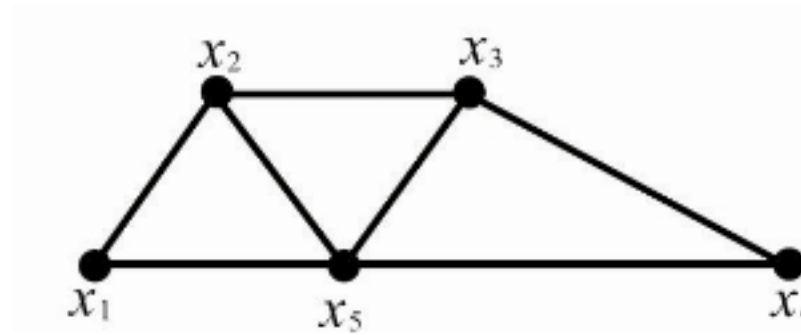
Независимые множества вершин и рёбер

- Множество вершин называется *независимым* (или ***внутренне устойчивым***), если никакие две из них не смежны.
- Наибольшее число вершин в независимом множестве вершин называется ***вершинным числом независимости*** и обозначается β_0 .

1. $\beta_0(K_p) = 1$, если K_p — полный граф.
2. $\beta_0(K_{m,n}) = \max(m, n)$, если $K_{m,n}$ — полный двудольный граф.
3. $\beta_0(\overline{K}_p) = p$, если $\overline{K}_p$ — вполне несвязный граф.
4. $\beta_0(G_1 \cup G_2) = \beta_0(G_1) + \beta_0(G_2)$, если G_1 и G_2 — компоненты несвязного графа.

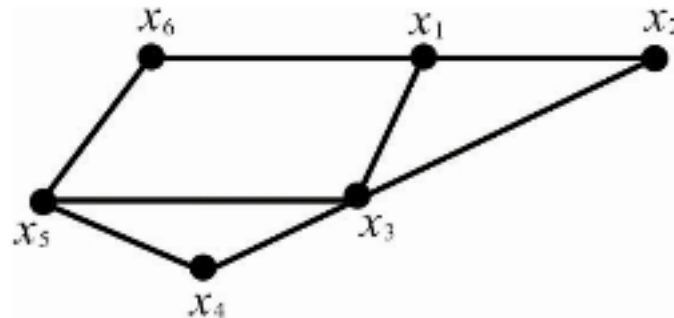
Ясно, что множество вершин S является независимым тогда и только тогда, когда $\forall v \in S \ (\Gamma(v) \cap S = \emptyset)$.

Пример внутренне устойчивых подмножеств



$$S_1 = \{x_1, x_3\}, S_2 = \{x_5\}, S_3 = \{x_2, x_4\}, S_4 = \{x_1, x_4\}.$$

- Определить максимальное количество источников, которые не влияют друг на друга.



$$\beta_0 = 3, S = \{x_2, x_4, x_6\}.$$

Независимые множества вершин и рёбер

- Множество рёбер называется *независимым* (или *паросочетанием*), если никакие два из них не смежны.
- Наибольшее число рёбер в независимом множестве рёбер называется *рёберным числом независимости* и обозначается β_1 .

1. $\beta_1(C_{2n}) = n$, если C_{2n} — чётный цикл.
2. $\beta_1(C_{2n+1}) = n - 1$, если C_{2n+1} — нечётный цикл.
3. $\beta_1(K_{2n}) = n$, если K_{2n} — полный граф с чётным числом вершин.
4. $\beta_1(K_{2n+1}) = n$, если K_{2n+1} — полный граф с нечётным числом вершин.
5. $\beta_1(K_{m,n}) = \min(m, n)$, если $K_{m,n}$ — полный двудольный граф.

Связь чисел независимости и покрытий

- Индексы 0 и 1 в обозначениях $\alpha_0, \alpha_1, \beta_0, \beta_1$ выбраны из следующих мнемонических соображений: индекс 0 соответствует вершинам, так как вершина нульмерна, а 1 — рёбрам, так как ребро одномерно.
- **ТЕОРЕМА.** Для любого нетривиального связного графа
$$\alpha_0 + \beta_0 = p = \alpha_1 + \beta_1$$
- *Условия связности и нетривиальности гарантируют, что все четыре инварианта определены. Хотя это условие является достаточным, оно не является необходимым. Например, для графа $K_n \cup K_n$ заключение теоремы остается справедливым, хотя условие не выполнено.*
- Задача отыскания экстремальных независимых и покрывающих множеств возникает во многих практических задачах. Например, пусть дано множество процессов, использующих *неразделяемые ресурсы*. Соединим рёбрами вершины, соответствующие процессам, которым требуется **один и тот же ресурс**. Тогда β_0 будет определять количество возможных **параллельных процессов**.

Постановка задачи отыскания наибольшего независимого множества вершин

- Задача отыскания наибольшего независимого множества вершин (и тем самым определения вершинного числа независимости β_0) принадлежит к числу трудоёмких.
- Эту задачу можно поставить следующим образом. Пусть задан граф $G(V, E)$. Найти такое множество вершин X , $X \subset V$, что

$$w(X) = \max_{Y \in \mathcal{E}} w(Y),$$

$$\text{где } w(Y) \stackrel{\text{Def}}{=} |Y|, \text{ а } \mathcal{E} \stackrel{\text{Def}}{=} \{Y \subset V \mid \forall u, v \in Y \ ((u, v) \notin E)\}$$

Если бы семейство $\mathcal{E}$ независимых множеств оказалось матроидом, то поставленную задачу можно было бы решить жадным алгоритмом. Однако семейство $\mathcal{E}$ матроидом не является. Хотя аксиомы M_1 и M_2 выполнены, так как пустое множество вершин независимо и всякое подмножество независимого множества независимо, но аксиома M_3 не выполнена. Рассмотрим полный двудольный граф $K_{n, n+1}$. Доли этого графа образуют (максимальные) независимые множества, и их мощности различаются на 1. Однако никакая вершина из большей доли не может быть добавлена к вершинам меньшей доли с сохранением независимости.

Поиск с возвратами

Вход: граф $G(V, E)$.

Выход: наибольшее независимое множество X .

$m := 0$ { наилучшее известное значение β_0 }

for $Y \in 2^V$ **do**

if $Y \in \mathcal{E}$ & $|Y| > m$ **then**

$m := |Y|$; $X := Y$ { наилучшее известное значение X }

end if

end for

- Для выполнения этого алгоритма потребуется $O(2^P)$ шагов.

Поиск с возвратами (*backtracking*)

- При решении переборных задач большое значение имеет способ организации перебора (в нашем случае — способ построения и последовательность перечисления множеств Y). Наиболее популярным является следующий способ организации перебора, основанный на идее поиска в глубину и называемый *поиском с возвратами*.
- Находясь в некоторой ситуации, пробуем изменить её допустимым образом в надежде найти решение. Если изменение не привело к успеху, то возвращаемся в исходную ситуацию (отсюда название .поиск с возвратами.) и пробуем изменить её другим образом, и так до тех пор, пока не будут перебраны все возможности.

Вход: граф $G(V, E)$.

Выход: наибольшее независимое множество X .

$m := 0$ { наилучшее известное значение β_0 }

$X := \emptyset$ { наибольшее известное независимое множество X }

$BT(\emptyset, V)$ { вызов рекурсивной процедуры BT }

Основная работа выполняется рекурсивной процедурой VT

Вход: S — текущее независимое множество вершин, T — оставшиеся вершины графа.

Выход: изменение глобальной переменной X , если текущее множество не может быть расширено (является максимальным).

```
for  $v \in T$  do
  if  $S + v \in \mathcal{E}$  then
    if  $|S| > m$  then
       $X := S; m := |S|$  { наибольшее известное независимое множество }
    end if
     $VT(S + v, T \setminus \Gamma^*(v))$  { пробуем добавить  $v$  }
  end if
end for
```

Проверить сохранение условия независимости нетрудно, например, с помощью следующей функции

Вход: независимое множество S и проверяемая вершина v .

Выход: **true**, если множество $S + v$ независимое, **false** — в противном случае.

```
for  $u \in S$  do
  if  $(u, v) \in E$  then
    return false { множество  $S + v$  зависимое }
  end if
end for
return true { множество  $S + v$  независимое }
```

Комментарий

- Последний цикл не включен в явном виде в рекурсивную процедуру ВТ, чтобы не загромождать основной текст и не затуманивать идею поиска с возвратами.
- Таким образом, множество S , а следовательно, и множество X — независимые.
- В тот момент, когда множество S нельзя расширить, оно максимально по определению.
- Переменная t глобальна, поэтому среди всех максимальных независимых множеств в конце работы алгоритма построенное множество X является наибольшим независимым множеством вершин.

Улучшенный перебор

- Применение метода поиска с возвратами не гарантирует эффективности — трудоёмкость поиска с возвратами имеет тот же порядок, что и другие способы перебора (в худшем случае).
- Используя конкретную информацию о задаче, в некоторых случаях можно существенно сократить трудоёмкость выполнения каждого шага перебора или уменьшить количество перебираемых возможностей в среднем (при сохранении оценки количества шагов в худшем случае).
- Такие приёмы называются методами улучшения перебора.
- Например, может оказаться, что некоторые варианты заведомо не могут привести к решению, а потому их можно не рассматривать.
- Рекурсивная форма метода поиска с возвратами удобна для понимания, но не является самой эффективной. По сути, рекурсия здесь используется для сохранения **контекста** — то есть информации, характеризующей текущий рассматриваемый вариант.
- Если использовать другие методы сохранения контекста, то поиск с возвратами может быть реализован без явной рекурсии, а значит, более эффективно.
- **Идея:** начинаем с пустого множества и пополняем его вершинами с сохранением независимости (пока возможно). Подробности см. учебник Новикова.

Алгоритм построения максимальных независимых множеств вершин

Вход: граф $G(V, E)$, заданный списками смежности $\Gamma[v]$.

Выход: последовательность максимальных независимых множеств.

$k := 0$ { количество элементов в текущем независимом множестве }

$S[k] := \emptyset$ { $S[k]$ — независимое множество из k вершин }

$Q^-[k] := \emptyset$ { $Q^-[k]$ — множество вершин, уже использованных для расширения $S[k]$ }

$Q^+[k] := V$

{ $Q^+[k]$ — множество вершин, которые можно использовать для расширения $S[k]$ }

$M1$: { шаг вперед }

select $v \in Q^+[k]$ { расширяющая вершина }

$S[k+1] := S[k] \cup \{v\}$ { расширенное множество }

$Q^-[k+1] := Q^-[k] \cup \Gamma[v]$ { вершина v использована для расширения }

$Q^+[k+1] := Q^+[k] \setminus (\Gamma[v] \cup \{v\})$

{ все вершины, смежные с v , не могут быть использованы для расширения }

$k := k + 1$

$M2$: { проверка }

for $u \in Q^-[k]$ **do**

if $\Gamma[u] \cap Q^+[k] = \emptyset$ **then**

goto $M3$ { можно возвращаться }

end if

end for

```

if  $Q^+[k] = \emptyset$  then
    if  $Q^-[k] = \emptyset$  then
        yield  $S[k]$  { множество  $S[k]$  максимально }
    end if
    goto  $M3$  { можно возвращаться }
else
    goto  $M1$  { можно идти вперед }
end if
 $M3$  : { шаг назад }
 $v := \text{last}(S[k])$  { последний добавленный элемент }
 $k := k - 1$ 
 $S[k] := S[k + 1] - \{v\}$ 
 $Q^-[k] := Q^-[k] \cup \{v\}$  { вершина  $v$  уже добавлялась }
 $Q^+[k] := Q^+[k] \setminus \{v\}$ 
if  $k = 0$  &  $Q^+[k] = \emptyset$  then
    stop { перебор завершён }
else
    goto  $M2$  { переход на проверку }
end if

```

Максимальное независимое множество вершин в дереве



Стандартный обход дерева (обратный)

TREEMIS(v):

$skipv \leftarrow 0$

for each child w of v

$skipv \leftarrow skipv + \text{TREEMIS}(w)$

$keepv \leftarrow 1$

for each grandchild x of v

$keepv \leftarrow keepv + x.MIS$

$v.MIS \leftarrow \max\{keepv, skipv\}$

return $v.MIS$

TREEMIS2(v):

$v.MISno \leftarrow 0$

$v.MISyes \leftarrow 1$

for each child w of v

$v.MISno \leftarrow v.MISno + \text{TREEMIS2}(w)$

$v.MISyes \leftarrow v.MISyes + w.MISno$

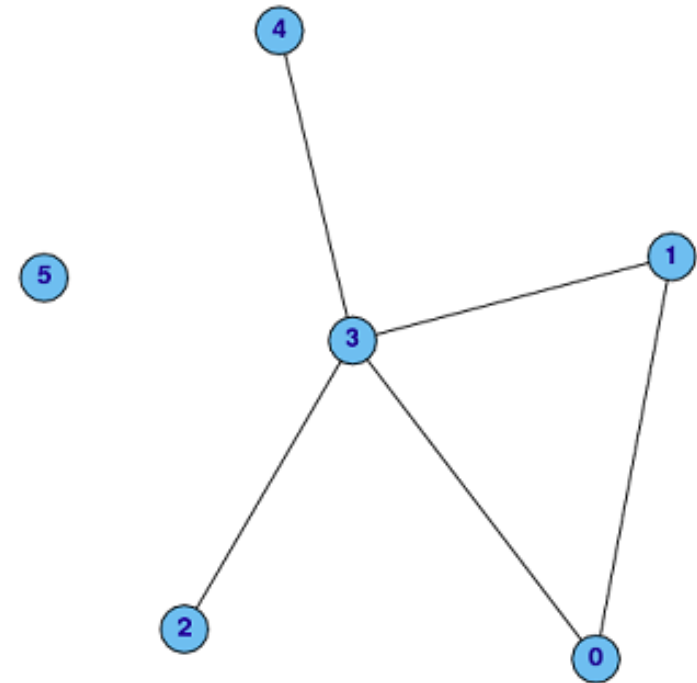
return $\max\{v.MISyes, v.MISno\}$

$O(n)$

Приближенный алгоритм решения задачи о вершинном покрытии

- Выбрать в неориентированном графе минимальное (по количеству вершин) вершинное покрытие.
- Построить простой алгоритм решения этой задачи, ошибающийся не более чем в два раза.
- Это означает, что если есть оптимальное решение — множество вершин T , то полученное нами решение S удовлетворяет неравенству $|S| \leq 2|T|$.
- Вершинное покрытие — это такое множество S вершин графа, что у каждого ребра графа хотя бы один конец входит в S .

Пример



- Вершинным покрытием является, например, множество вершин **{0, 1, 2, 4, 5}**.
- Все шесть вершин графа также образуют вершинное покрытие. Однако, минимальным вершинным покрытием в рассматриваемом графе будет множество, состоящее всего из двух вершин. Например, **{1, 3, 5}**.

Алгоритм

- На каждом шаге алгоритма производим следующие действия:
Выбираем случайное ребро графа $e=(u, v)$.
- Добавляем в решение S обе выбранные вершины u и v .
- Удаляем из графа все ребра, инцидентные вершинам u или v .
- Повторяем этот шаг до тех пор, пока не удалим все ребра графа.

Алгоритм ошибается не более, чем в 2 раза.

Все рассматриваемые алгоритмом ребра e не имеют общих вершин. Следовательно, из каждого из этих рёбер в оптимальное решение T должна входить хотя бы одна вершина. Это значит, что $2|T| \geq |S|$.

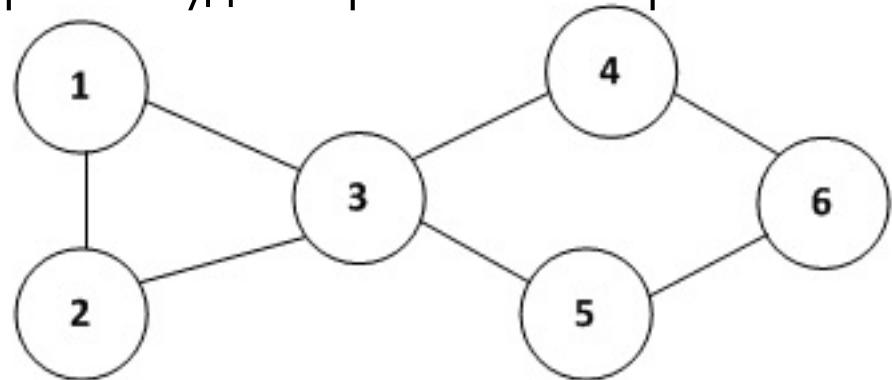
Пример работы алгоритма

Первая итерация:

- Выбираем случайное ребро. Например, ребро (1, 3).
- Добавляем в решение S обе вершины выбранного ребра: $S=\{1, 3\}$.
- Удаляем из графа все ребра, инцидентные вершинам 1 или 3.

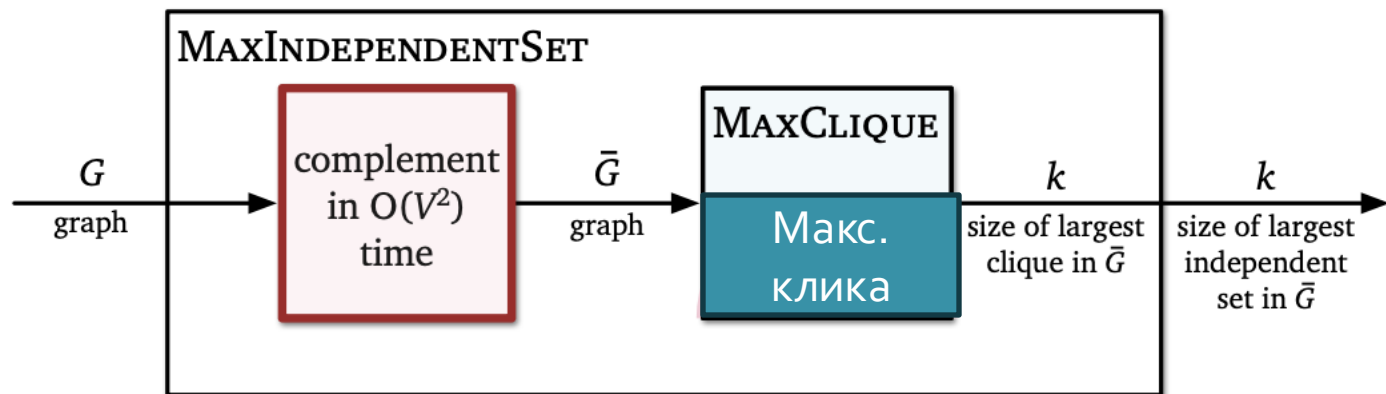
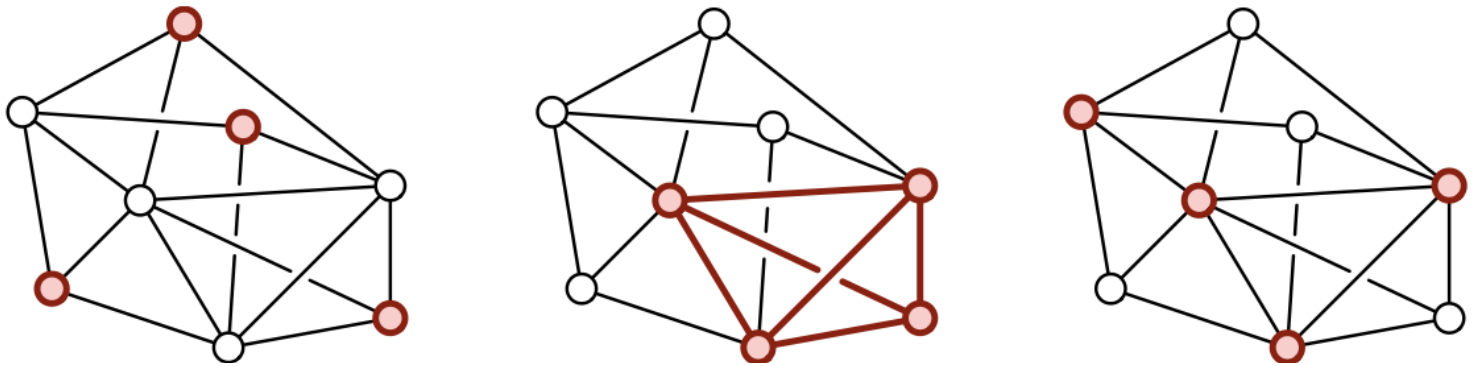
Вторая итерация:

- Выбираем случайное ребро. Пусть это будет ребро (4, 6).
- Добавляем в решение S обе вершины выбранного ребра: $S=\{1, 3, 4, 6\}$.
- Удаляем из графа все ребра, инцидентные вершинам 4 или 6.
- В графе не осталось ребер. Следовательно, результатом работы нашего алгоритма будет вершинное покрытие $S=\{1, 3, 4, 6\}$.

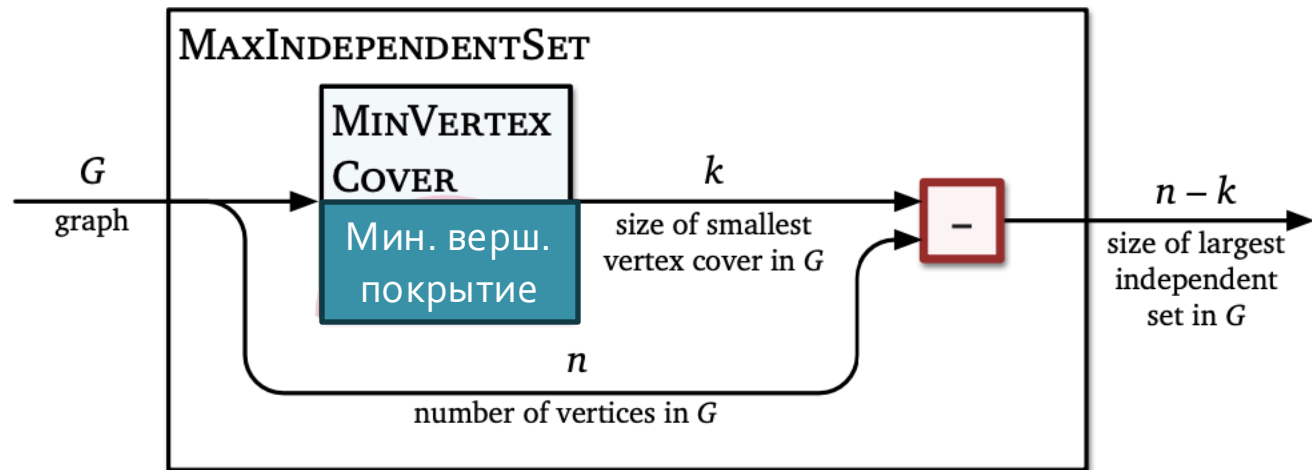


Клика и вершинное покрытие

- Найти размер наименьшего вершинного покрытия в заданном графе.
- Множество вершин является независимым в графе G , если и только если те же вершины определяют клику в $\bar{G}$.



Клика и вершинное покрытие



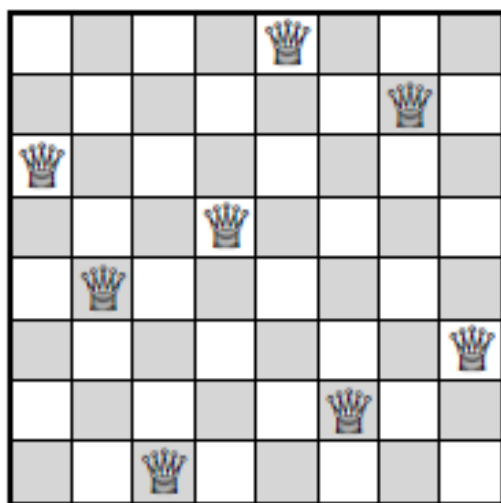
Латинский квадрат

- Задача известна еще с древности, необходимо расставить буквы таким образом, чтобы ни в одной строке и ни в одной колонке не было одинаковых.
- Само название **Латинский квадрат** получило из-за привычки использовать буквы латинского Леонардом Эйлером для данной задачи.
- Из *латинского квадрата* (и ряда похожих задач) естественным образом появлялись новые, такие как задача о ферзях, где добавляется дополнительное ограничение на диагонали.

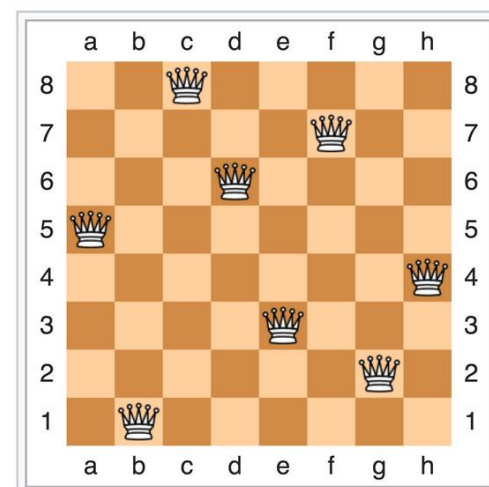
A	B	D	C
B	C	A	D
C	D	B	A
D	A	C	B

Задача о восьми ферзях

- Расставить на шахматной доске 8 ферзей так, чтобы они не атаковали друг друга.
- Является задачей об отыскании максимальных независимых множеств.
- Достаточно представить доску в виде графа с 64 вершинами (соответствующие клеткам доски), которые смежны, если клетки находятся на одной вертикали, горизонтали или диагонали.



Первое решение
Гаусса задачи о
восьми ферзях,
представленное в
виде массива
[5, 7, 1, 4, 2, 8, 6, 3]



[?, ?, ?, ?, ?, ?, ?, ?]

Постановка задачи

- Задачу придумал в 1848 году Макс Беццель (под псевдонимом Schachfreund – шахматный друг). В 1869 году Франсуа-Жозеф Юсташ Лионетт предложил обобщенную задачу $n \times n$ клеток.
- С тех пор многие математики (например, Карл Фридрих Гаусс), работали над задачей. В 1850 году он писал в письме к другу Г. Шумахеру о том, что задача о 8 ферзях имеет 92 решения («искать на ощупь»). Позднее алгоритмисты и программисты, такие как Дейкстра, придумали множество подходов к поиску и подсчету решений.
- **Найти:** расстановку максимально возможного числа ферзей.
- На вход число – размер доски, а на выход позиции ферзей (одного решения).

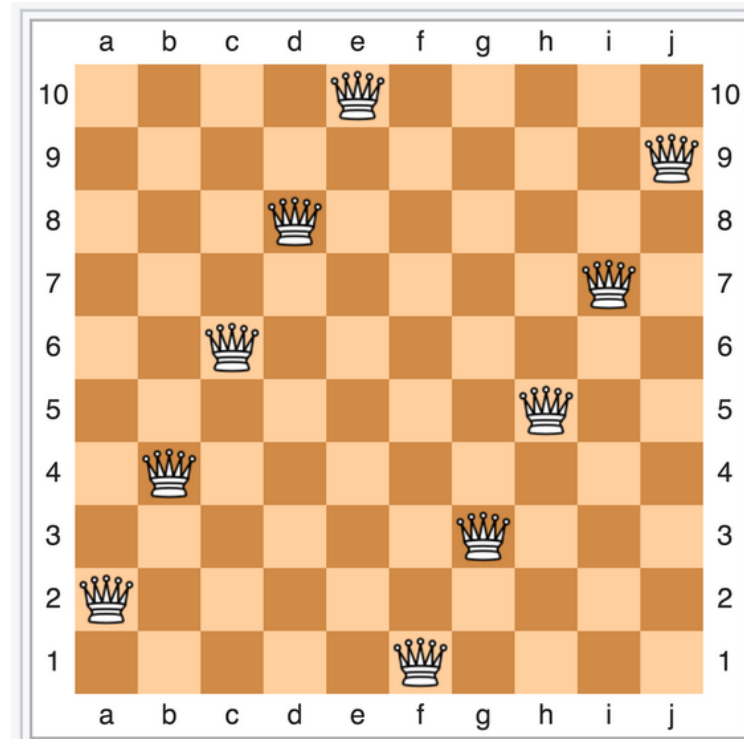
- **Подсчет числа решений**

Задача ставится достаточно просто:

Дано: размер пустой доски N

Найти: N – число возможных расстановок N ферзей на доске

Например, размер доски $N = 1$, тогда число возможных расстановок $N = 1$.
 $N = 8 \Rightarrow N = 92$.



Алгоритм Гаусса и Лакьера для задачи с n ферзями

PLACEQUEENS($Q[1..n], r$):

if $r = n + 1$

print $Q[1..n]$

else

for $j \leftarrow 1$ to n

legal $\leftarrow$ TRUE

for $i \leftarrow 1$ to $r - 1$

if $(Q[i] = j)$ or $(Q[i] = j + r - i)$ or $(Q[i] = j - r + i)$

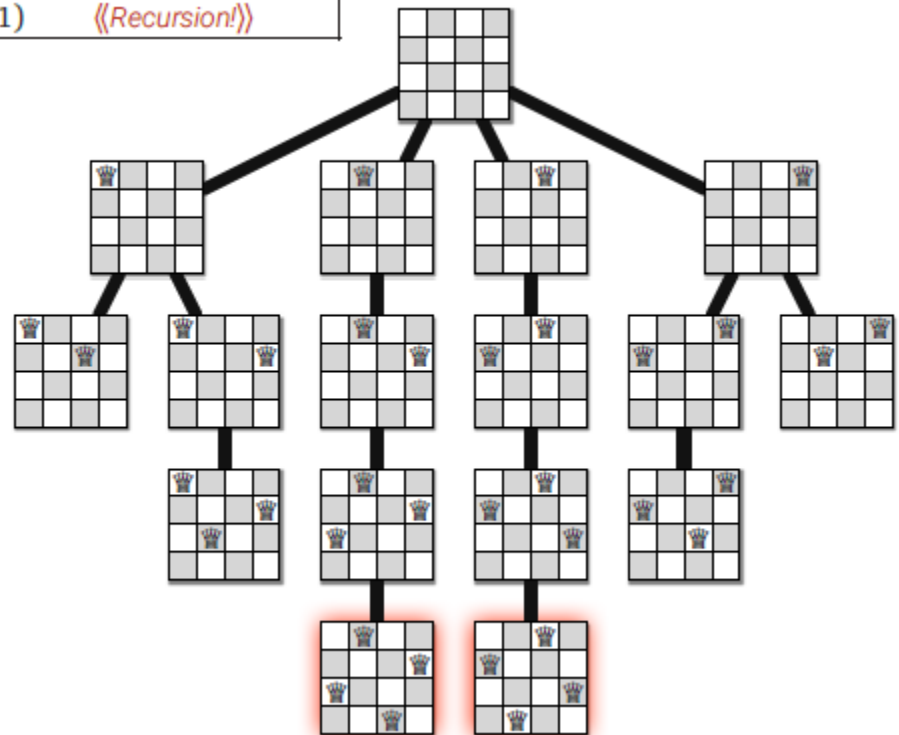
legal $\leftarrow$ FALSE

if legal

$Q[r] \leftarrow j$

PLACEQUEENS($Q[1..n], r + 1$)

⟨⟨Recursion!⟩⟩

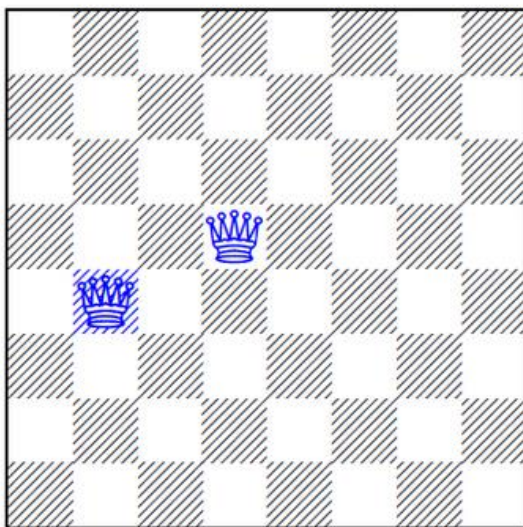


Дополнение до N ферзей

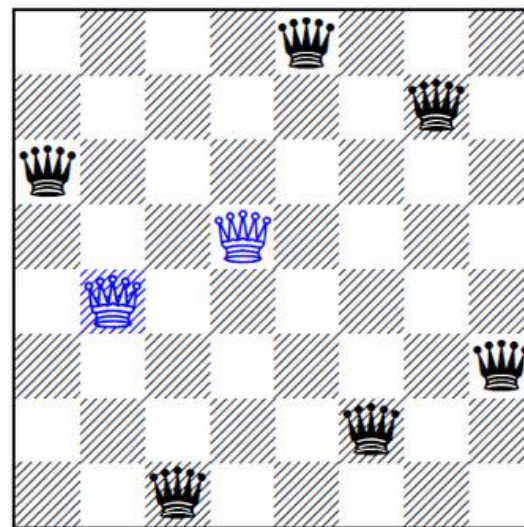
- **Дано:** размер доски N и M позиций уже установленных ферзей

Найти: позиции оставшихся $N - M$ ферзей

Дано:



Найти:



Минимальное и наименьшее доминирующее множество

- Множество вершин $S \subseteq V$ графа $G(V, E)$ называется **доминирующим множеством** (или *внешне устойчивым*), если $S \cup \Gamma(S) = V$, то есть

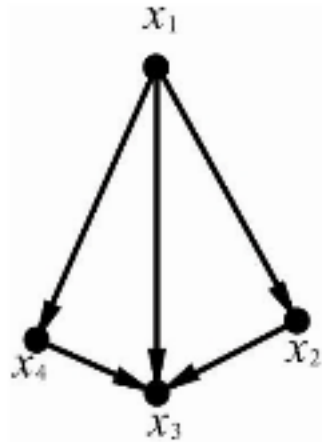
$$\forall v \in V (v \in S \vee \exists s \in S ((s, v) \in E))$$

- Очевидно, что множество S доминирует тогда и только тогда, когда $\forall v \notin S (\Gamma(v) \cap S \neq \emptyset)$

что равносильно утверждению $\forall v \in V (\exists s \in S (d(v, s) \leq 1))$

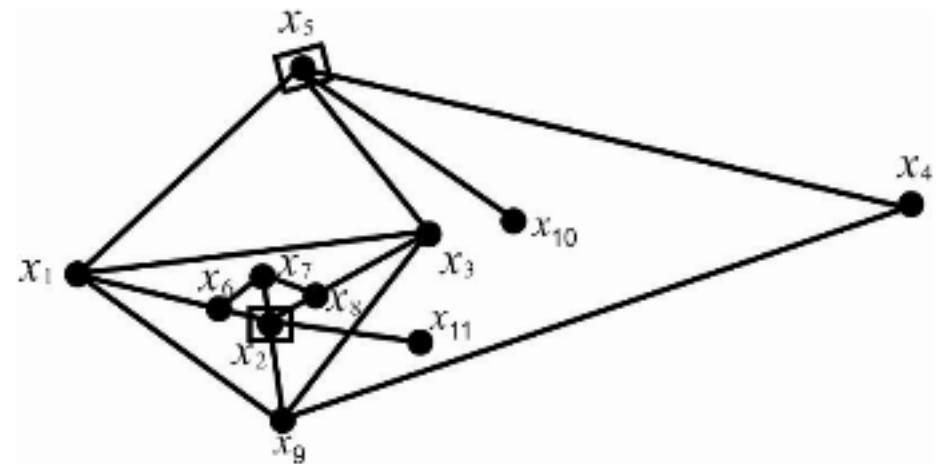
- Доминирующее множество называется *минимальным*, если никакое его подмножество не является доминирующим.
- Доминирующее множество называется *наименьшим*, если число элементов в нём наименьшее возможное.

Пример внешне устойчивого подмножества



$R_1 = \{x_1, x_3\}$, $R_2 = \{x_1, x_3, x_4\}$, $R_3 = \{x_3\}$

Какое минимальное количество видеокamer необходимо установить, чтобы держать под наблюдением 11 объектов

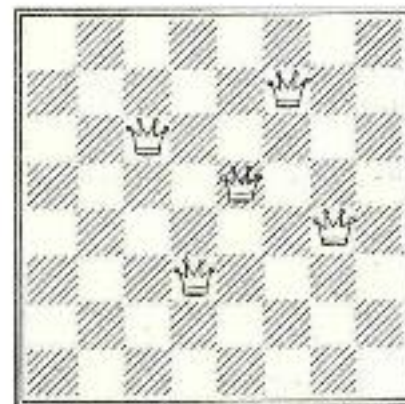


- Наименьшее из количеств вершин в подмножествах R_i называется числом внешней устойчивости $\delta(G)$ (числом доминирования) графа G .

Задача о пяти ферзях

- Расстановка 5 ферзей на шахматной доске, при которой каждое поле будет находиться под ударом одного из них.
- Является задачей об отыскании наименьших доминирующих множеств.

Задача обхода шахматной доски конём
Требуется, поставив коня на любое поле доски («первый ход»), последовательно пройти им все поля, не занимая ни одно из них дважды. Если после этого 65-м ходом конь может попасть на исходное поле, маршрут называется замкнутым.

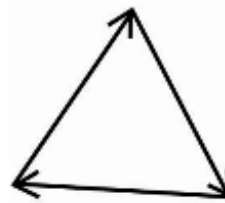


Доминирование и независимость

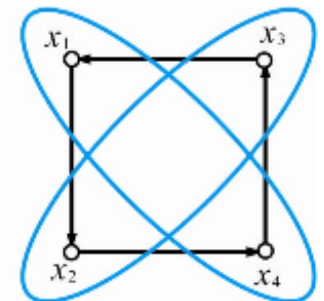
- ТЕОРЕМА *Независимое множество вершин является максимальным тогда и только тогда, когда оно является доминирующим.*
- [$= >$] Пусть множество вершин $S \subset V$ — максимальное независимое.
Допустим (от противного), что оно не доминирующее.
Тогда существует вершина v , находящаяся на расстоянии больше 1 от всех вершин множества S . Эту вершину можно добавить к S с сохранением независимости, что противоречит максимальности.
- [$< =$] Пусть S — независимое доминирующее множество.
Допустим (от противного), что оно не максимальное.
Тогда существует вершина v , не смежная ни с одной из вершин множества S , то есть находящаяся на расстоянии больше 1 от всех вершин множества S . Это противоречит тому, что множество S — доминирующее.

Ядро графа

- Независимое доминирующее множество вершин называется ядром графа.
- В полном графе K_p каждая из p вершин является ядром и других ядер нет.
- Следствие теоремы. **Любой граф имеет ядро.**
- Понятия независимости, доминирования и ядра применимы как к графам, так и к орграфам.
- Существуют орграфы, не имеющие ядра. Например, контур C_3 не имеет ядра.



G_1
 $\beta_0=1, \delta=2$



G_2
 $\{x_1, x_3\}, \{x_2, x_4\}$

Алгоритм, строящий некоторое ядро S

```
 $S := \emptyset$  { ядро }  
while  $V \neq \emptyset$  do  
  select  $v \in V$  { любая нерассмотренная вершина }  
   $S := S + v$  { расширяем множество  $S$  }  
   $V := V \setminus \Gamma^*(v)$  { удаляем вершины, которые не могут быть использованы для  
    расширения }  
end while
```



- Задача выделения ядра в произвольном орграфе оказывается NP-полной.
- Задача нахождения ядер графа возникает, например, в теории игр для определения стратегии игры, теории управления, радиоэлектронике и других науках.
- Для нахождения ядер графа можно применять алгоритм Магу, использующий булеву алгебру для анализа матрицы смежности графа.

Свойства ядра графа

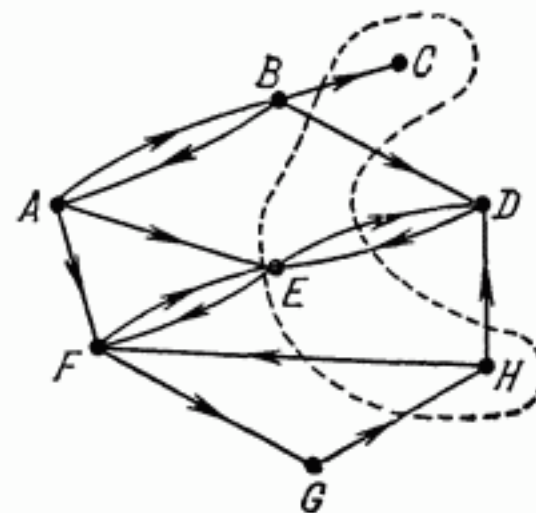
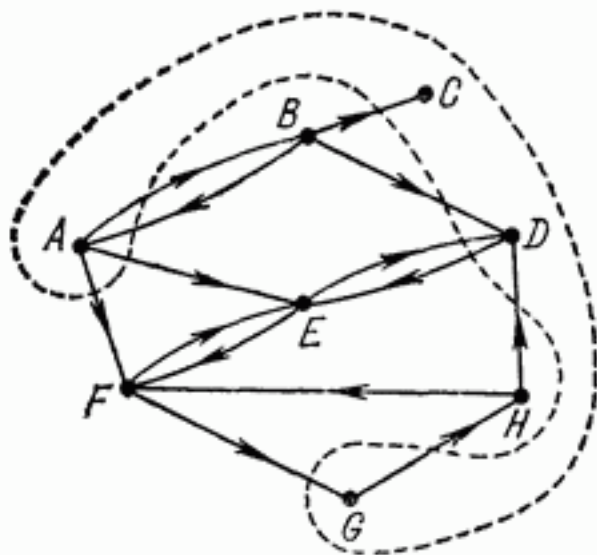
- Ядро не может содержать петель или смежных вершин
- Максимальное внутренне устойчивое подмножество.
- Одновременно максимальное внутренне устойчивое и минимальное внешне устойчивое.
- Граф без контуров всегда обладает ядром.
- Каждый орграф, не имеющий контуров нечетной длины, обладает ядром.
- Граф, не имеющий контуров нечетной длины, допускает ядро.
- Рассмотрим булеву функцию на множестве A

$$F(A, X_i) = \begin{cases} 0, & \text{если } X_i \notin A, \\ 1, & \text{если } X_i \in A. \end{cases}$$

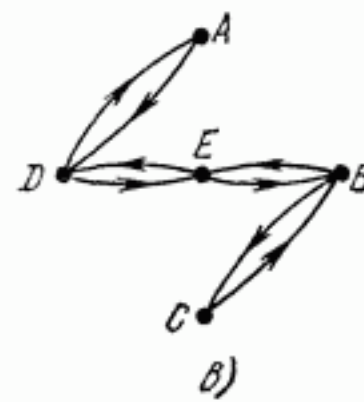
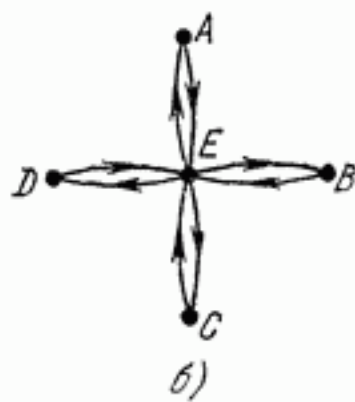
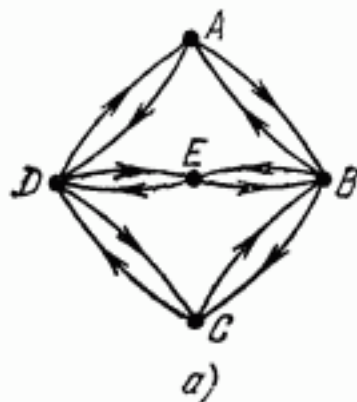
$$F(A, X_i) = 1 - \max_{X_j \in rX_i} F(A, X_j)$$

- Условие необходимо и достаточно, чтобы A являлось ядром графа $G(V, E)$, $A \subseteq V$
- При этом $F(A, \emptyset) = 0$.

Ядро графа. Примеры



$N_1 = \{A, C, D, G\}$ и $N_2 = \{C, E, H\}$.



Алгоритм Магу для определения множества внутренней устойчивости графа

- 1. Для графа составляется матрица смежности.
- 2. По таблице смежности выписываются все парные дизъюнкции.
- 3. Выражение приводится к ДНФ.
- 4. Для любой элементарной конъюнкции выписываются недостающие элементы, которые и образуют множество внутренней устойчивости.

Алгоритм Магу для определения множества внутренней устойчивости графа

Пусть дан граф $G = \langle V, E \rangle$. Для данного графа существует множество внутренней устойчивости X^* .

Введем булеву переменную x_i , которая определяется следующим образом:

$x_i = 1$, если вершина x_i принадлежит множеству внутренней устойчивости $x_i \in X^*$;

$x_i = 0$, если вершина x_i не принадлежит множеству внутренней устойчивости $x_i \notin X^*$;

Введем булеву переменную α_{ij} :

$\alpha_{ij} = 1$, если между i -той и j -той вершиной есть дуга;

$\alpha_{ij} = 0$, если между i -той и j -той вершиной нет дуги.

$$\forall i, j \in V \{ (x_i \& \alpha_{ij}) \rightarrow \overline{x_j} \} = 1$$

$$\alpha_{ij} = 1 \& (\overline{x_i} \vee \overline{x_j}) = 1$$

Пример

Для всех единиц
выписываются парные
дизъюнкции:

$$(x_1 \vee x_2)(x_1 \vee x_3)(x_1 \vee x_4)(x_2 \vee x_1)(x_3 \vee x_1)(x_4 \vee x_3)$$

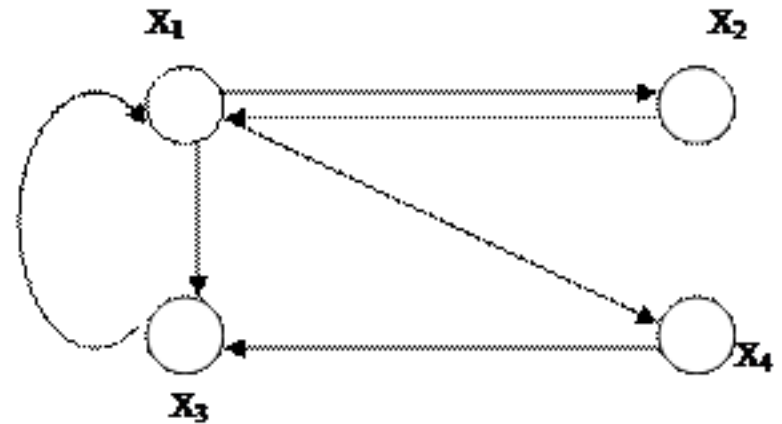
$$(x_1 \vee x_2)(x_1 \vee x_3)(x_1 \vee x_4)(x_2 \vee x_1)(x_3 \vee x_1)(x_4 \vee x_3) =$$

$$= (x_1 \vee x_2)(x_1 \vee x_3)(x_1 \vee x_4)(x_4 \vee x_3) =$$

$$= (x_1 \vee x_2 x_3)(x_4 \vee x_1 x_3) =$$

$$= x_1 x_4 \vee x_1 x_3 \vee x_2 x_3 x_4 \vee x_1 x_2 x_3 =$$

$$= x_1 x_4 \vee x_1 x_3 \vee x_2 x_3 x_4$$



Приведем выражение к ДНФ:

Множества внутренней устойчивости: $\{x_2, x_3\}$, $\{x_2, x_4\}$, $\{x_1\}$

Числом внутренней устойчивости: $\alpha=2$.

Алгоритм Магу для определения множества внешней устойчивости

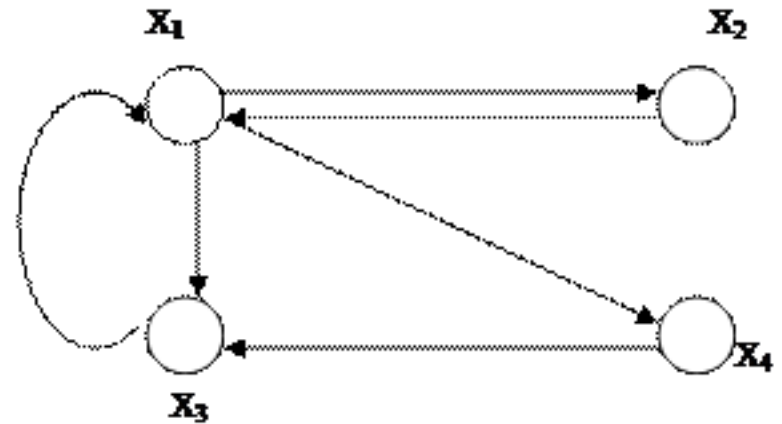
- 1. Для графа составляется матрица смежности
- 2. Матрица смежности дополняется единицами (1) по главной диагонали.
- 3. Для каждой строки выписываются дизъюнкции.
- 4. Выражение приводится к ДНФ.
- 5. Все вершины, входящие в элементарную конъюнкцию, образуют множество внешней устойчивости

$$\bigwedge_{i=1}^n (x_i \vee \bigvee_{j=1}^n \alpha_{ij} \wedge x_j) = 1$$

$$\bigwedge_{i=1}^n (x_i \vee \bigvee_{\alpha_{ij}=1} x_j) = 1$$

Пример

Для каждой строки записываются дизъюнкции:



$$(x_1 \vee x_2 \vee x_3 \vee x_4)(x_1 \vee x_2)(x_1 \vee x_3)(x_3 \vee x_4)$$

Приведем выражение к ДНФ:

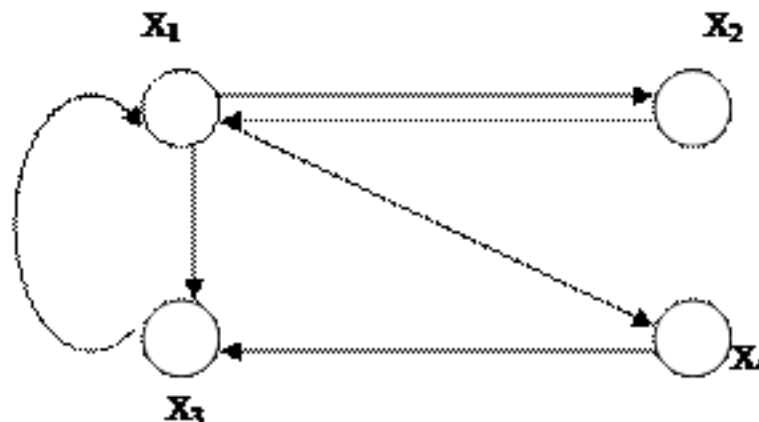
$$\begin{aligned}
 &(x_1 \vee x_2 \vee x_3 \vee x_4)(x_1 \vee x_2)(x_1 \vee x_3)(x_3 \vee x_4) = \\
 &= (x_1 \vee x_2)(x_1 \vee x_3)(x_3 \vee x_4) = (x_1 \vee x_2 x_3)(x_3 \vee x_4) = \\
 &= x_1 x_3 \vee x_1 x_4 \vee x_2 x_3 \vee x_2 x_3 x_4 = x_1 x_3 \vee x_1 x_4 \vee x_2 x_3
 \end{aligned}$$

Множества внешней устойчивости:

$$\{x_1, x_3\}, \{x_1, x_4\}, \{x_2, x_3\}$$

Числом внешней устойчивости: $\beta=2$.

Пример ядра графа



- **Ядром графа** называется подмножество вершин, являющихся одновременно внутренней и внешней устойчивостью.

Множества внутренней устойчивости:

$\{x_1, x_3\}$, $\{x_1, x_4\}$, $\{x_2, x_3\}$

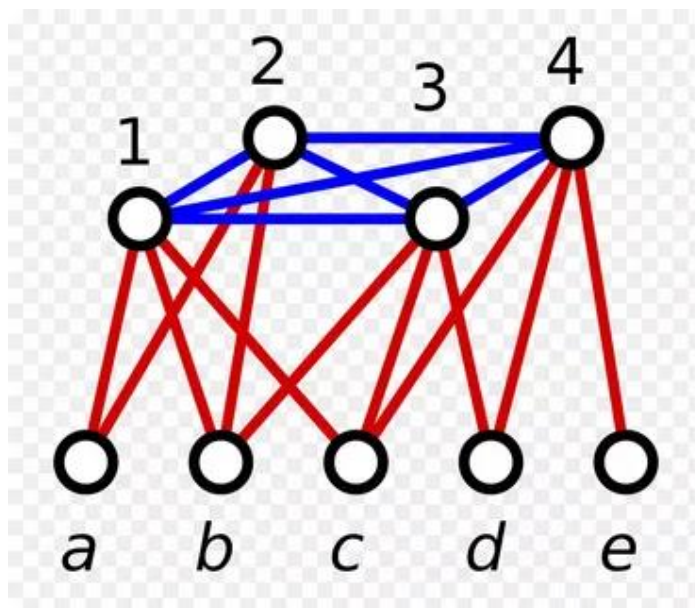
Множества внешней устойчивости:

$\{x_2, x_3\}$, $\{x_2, x_4\}$, $\{x_1\}$

Граф имеет ядро $\{x_2, x_3\}$

Задача о наименьшем покрытии

- Задача о наименьшем покрытии (сокращённо ЗНП) является примером общей экстремальной задачи, к которой прямо или косвенно сводятся многие практические задачи.
- Эта задача является классической, хорошо изучена и часто используется в качестве теста для сравнения и оценки различных общих методов решения трудоёмких задач.



Задача о наименьшем покрытии

Пусть каждой вершине сопоставлена некоторая цена. Требуется выбрать доминирующее множество с наименьшей суммарной ценой.

- **Задача о выборе переводчиков.** Организации нужно нанять переводчиков для перевода с определённого множества языков. Каждый из имеющихся переводчиков владеет некоторыми иностранными языками и требует определённую зарплату. Требуется определить, каких переводчиков следует нанять, чтобы сумма расходов на зарплату была минимальной. Задача о выборе переводчиков сводится к ЗНП следующим образом.
- Рассмотрим полный двудольный граф с долями V_1 и V_2 , где доля V_1 соответствует множеству переводчиков, а доля V_2 — множеству языков. Вершины v_1 и v_2 смежны, если переводчик v_1 владеет языком v_2 . Требуется найти наименьшее покрывающее множество $S \subseteq V_1$.

Задача о переводчиках

<i>Язык</i>	<i>Переводчик</i>				
	<i>A</i>	<i>B*</i>	<i>C*</i>	<i>D*</i>	<i>E</i>
Французский	1	0	1	1	0
Немецкий	1	1	0	0	0
Греческий	0	1	0	0	0
Итальянский	1	0	0	1	0
Испанский	0	0	1	0	0
Русский	0	1	1	0	1
Китайский	0	0	0	1	1

Задача о наименьшем покрытии

Пусть каждой вершине сопоставлена некоторая цена. Требуется выбрать доминирующее множество с наименьшей суммарной ценой.

- **Задача о развозке.** Поставщику нужно доставить товары своим потребителям. Имеется множество возможных маршрутов, каждый из которых позволяет обслужить определённое подмножество потребителей и требует определённых расходов.
- Требуется определить, какие маршруты следует использовать, чтобы все потребители были обслужены, а сумма транспортных расходов была минимальной.
- Задача о развозке сводится к ЗНП аналогичным образом.
- Здесь V_1 — множество маршрутов, V_2 - множество потребителей и вершины v_1 и v_2 смежны, если маршрут v_1 обслуживает потребителя v_2 .

Другие задачи

- Информационный поиск

Предположим, что некоторое количество единиц информации хранится в N массивах длины c_j , $j = 1, 2, \dots, N$, причем на каждую единицу информации отводится по меньшей мере один массив. В некоторый момент делается запрос о M единицах информации. Они могут быть получены различными способами при помощи поиска в массиве. Для того чтобы получить все M единиц информации и при этом произвести просмотр массивов наименьшей длины, надо решить ЗНП, в которой элемент t_{ij} матрицы T равен 1, если информация i находится в массиве, и 0 в противном случае.

Другие задачи

- Маршруты полетов самолетов

Предположим, что вершины неориентированного графа G представляют аэропорты, а дуги графа G — этапы полетов (беспосадочные перелеты), которые осуществляются в заданное время. Любой маршрут в этом графе (удовлетворяющий ряду условий, которые могут встретиться на практике) соответствует некоторому реально выполнимому маршруту полета. Пусть имеется N таких возможных маршрутов и для каждого из них каким-то способом подсчитана его стоимость (например, стоимость j -го маршрута равна c_j). Задача нахождения множества маршрутов, имеющего наименьшую суммарную стоимость и такого, что каждый этап полета содержится хотя бы в одном выбранном маршруте, является задачей о наименьшем покрытии с матрицей $T = [t_{ij}]$, в которой элемент t_{ij} равен 1, если i -й этап содержится в j -м маршруте, и равен 0 в противном случае.

Задача о наименьшем покрытии

В матричной форме, упомянутой ранее, когда строки $(M \times N)$ -матрицы $[t_{ij}]$, состоящей из нулей и единиц, покрываются столбцами, ЗНП может быть сформулирована как задача линейного программирования:

$$\text{минимизировать } z = \sum_{j=1}^N c_j \xi_j$$

при ограничениях

$$\sum_{j=1}^N t_{ij} \xi_j \geq 1, \quad i = 1, 2, \dots, M, \quad (3.14)$$

где $c_j \geq 0$,

$$\xi_j = \begin{cases} 1, & \text{если } S_j \in \mathcal{S}', \\ 0, & \text{если } S_j \notin \mathcal{S}' \end{cases}$$

и

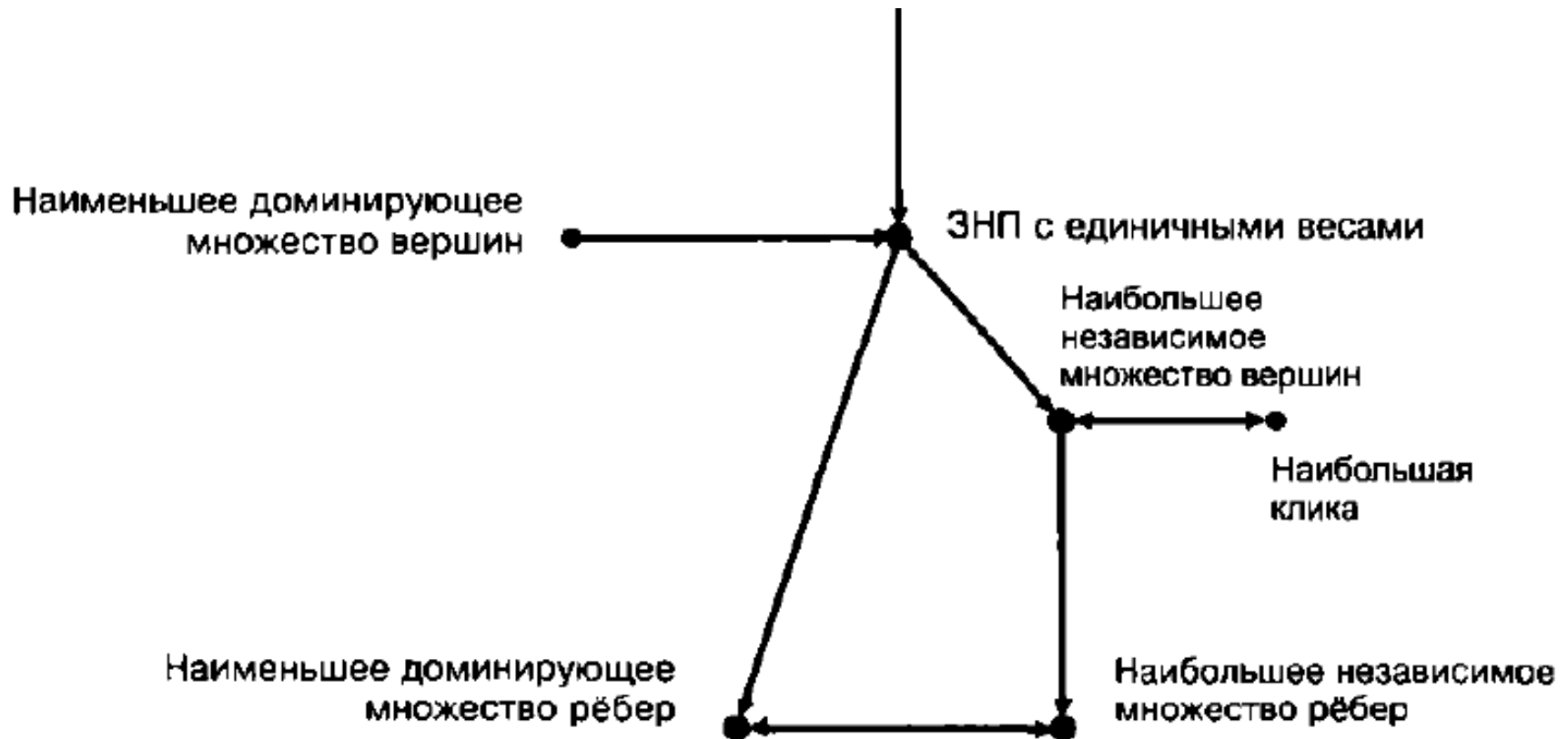
$$t_{ij} = \begin{cases} 1, & \text{если } r_i \in S_j, \\ 0, & \text{если } r_i \notin S_j. \end{cases}$$

Для ЗНР неравенства (3.14) обращаются в равенства

$$\sum_{j=1}^N t_{ij} \xi_j = 1, \quad i = 1, 2, \dots, M. \quad (3.15)$$

Связь задачи о наименьшем покрытии с другими задачами

Пример Пусть имеется конечное множество $V = \{v_1, \dots, v_p\}$ и семейство подмножеств этого множества $E = \{E_1, \dots, E_p\}$. Каждому подмножеству E_i приписан вес. Найти покрытие E' ($E' \subset E$) наименьшего веса. На языке графов v_i — это вершина, а E_i — множество смежности вершины, то есть $E_i := \Gamma^*(v_i)$.





СПАСИБО
ЗА ВНИМАНИЕ